Lab 5: Static Code Analysis — Issues Report

Program: pet_shelter.py | Animal Shelter Adoption Register

NAME : JAYASHREE D

SRN : PES2UG24CS203

SECTION : D

1. Overview

The original pet_shelter.py program was reviewed using Pylint, Bandit, and Flake8. The tools identified maintainability, style, exception-handling, and security concerns. The code was refactored to address the reported findings while preserving the intended shelter-register demonstration.

2. Issues Identified and Fixes Applied

Issue	Type	Original line(s)	Description	Fix approach
SQL query built using string interpolation	Security — Bandit B608	10	Values were inserted directly into an SQL statement, creating a possible SQL-injection risk.	Replaced interpolation with a parameterized SQL query using placeholders and a tuple of values.
Mutable default argument	Correctness / maintainability	9	The default tags=[] list could be reused across calls, causing values to leak between calls.	Changed the default to None and created a new list inside the function when no tags are supplied.
General-purpose random number generator	Security / quality — Bandit B311	22	random.randint is intended for general-purpose randomness, not security-sensitive codes.	Used secrets.randbelow to generate the four-digit adoption code.
CamelCase function names	Style — Pylint	9, 16, 22, 26, 34, 41	Names such as registerPet and oldestAvailable did not follow Python's snake_case convention.	Renamed functions to register_pet, oldest_available, adoption_code, clear_unadopted, and shelter_status.
Missing module and function docstrings	Documentation — Pylint	1 and function definitions	The original module and functions lacked docstrings explaining their purpose.	Added a module docstring and concise docstrings for each function.
Global variable modified inside a function	Maintainability — Pylint	17	The adopt function changed adoption_count through a global statement, coupling it to global state.	Passed the count into adopt and returned the updated count to the caller.
Bare except clause	Error handling — Flake8 E722 / Pylint	29	A bare except could catch unexpected exceptions and hide the cause of failure.	Caught sqlite3.Error specifically in the database operation.
Exception silently ignored	Reliability — Bandit B110	35–36	The original clear operation swallowed every Exception without reporting the failure.	Caught sqlite3.Error and printed a concise diagnostic message.
Long lines	Style — Flake8 E501 / Pylint	5, 10, 27	Long lines reduced readability and triggered line-length warnings.	Split SQL statements and function calls across multiple lines.
Multiple statements on one line	Style — Flake8 E701 / Pylint	41	The one-line if/return statement made control flow harder to read.	Expanded the conditional and return into separate, properly indented lines.
Percent-based SQL string formatting	Style / security	10	Percent formatting inserted values directly into the SQL query.	Removed percent formatting and used SQL parameters, improving readability and safety.
Module-level database connection	Design / testability	4–5 and database functions	A module-level connection made functions depend on shared external state and harder to test independently.	Introduced create_connection and passed the connection explicitly to database-related functions.

3. Verification After Refactoring

Check	Command	Result
Program execution	python pet_shelter.py	Executed successfully; printed the oldest available pet, a generated adoption code, and shelter status.
Pylint	python -m pylint pet_shelter.py	10.00/10.
Bandit	python -m bandit -r pet_shelter.py	No issues identified.
Flake8	python -m flake8 pet_shelter.py	No output; no violations reported.

4. Conclusion

This lab showed that static analysis helps detect potential problems early, although clean reports do not guarantee that software is completely free of bugs or security risks.

The final checks reported a Pylint score of 10.00/10, no Bandit issues, and no Flake8 violations.

After refactoring, the program ran successfully and retained its intended functionality.

Exception handling was improved so database errors are handled specifically instead of being silently ignored.

The mutable default argument and global variable dependency were removed to make the program more reliable and easier to maintain.

The code was improved by using parameterized SQL queries, safer adoption-code generation, and clearer function names and docstrings.

The static analysis of pet_shelter.py identified security, style, and maintainability issues using Pylint, Bandit, and Flake8.